

W15D03

Metrics & Prometheus

17 Juni 2026

1. Konteks & Posisi dalam Observability

Sesi ini adalah lapisan ketiga dari fondasi observability yang sedang dibangun. Tiga lapisan ini saling melengkapi dan tidak bisa dibalik urutannya:

Layer	Fungsi	Tools
W15D01 — Structured Logging	Tahu APA yang terjadi	structlog / logging
W15D02 — Request ID Tracing	Tahu SIAPA & KAPAN	ContextVar + UUID
W15D03 — Metrics & Prometheus	Tahu SEBERAPA SERING & LAMA	prometheus-client

Logging memberi detail kejadian satu per satu — seperti buku catatan. Metrics memberi gambaran agregat dari ribuan kejadian — seperti dashboard angka di dinding. Keduanya dibutuhkan: metrics untuk **deteksi dan alerting**, logging untuk **investigasi**.

2. Analogi — Manajer Restoran

Bayangkan kamu manajer restoran dengan dua sistem pencatatan:

- **Buku log dapur** — setiap pesanan dicatat detail: jam masuk, chef yang masak, waktu selesai. Ini adalah logging. Kamu tahu detail tiap kejadian.
- **Dashboard di dinding** — angka yang terus diperbarui: total pesanan hari ini 247, rata-rata waktu masak 13 menit, meja yang menunggu 8. Ini adalah metrics.

Ketika restoran tiba-tiba lambat, kamu **lihat dashboard dulu** — 'waktu masak naik dari 13 menit ke 31 menit sejak jam 15.00'. Setelah tahu kapan masalahnya, baru buka buku log untuk investigasi detail. Inilah hubungan Logging dan Metrics: metrics untuk deteksi, logging untuk investigasi.

3. Cara Kerja Prometheus

Prometheus adalah sistem monitoring open-source berbasis **pull model** (scraping). Berbeda dari sistem push (aplikasi kirim data ke server monitoring), Prometheus yang aktif mendatangi aplikasimu setiap N detik dan mengambil snapshot metric dari endpoint **/metrics**.

Komponen	Peran
FastAPI App + /metrics	Ekspos data metric dalam format teks Prometheus
Prometheus Server	Scrape /metrics tiap ~15 detik, simpan ke time-series DB
Time-Series Database	Disk lokal Prometheus — menyimpan semua nilai historis

Grafana	Visualisasi dari TSDB via query PromQL
Alertmanager	Kirim notifikasi (Slack, email, PagerDuty) saat threshold terlampaui

Kamu hanya bertanggung jawab pada satu hal: mengekspos data yang benar di `/metrics`. Prometheus, TSDB, Grafana, dan Alertmanager adalah urusan *infrastructure team* — atau bisa dijalankan lokal dengan `docker-compose` untuk *development*.

4. Empat Tipe Metric Prometheus

Ini adalah vocabulary dasar yang wajib dikuasai. Salah pilih tipe = data yang tidak berguna.

Tipe	Sifat	Analogi Restoran	Contoh Metric
Counter	Hanya naik, tidak pernah turun. Reset jika server restart	Total pesanan yang sudah masuk	<code>http_requests_total</code> , <code>predictions_total</code>
Gauge	Bisa naik dan turun bebas. Representasi yang dinamis	Meja yang sedang terisi sekarang	<code>active_connections</code> , <code>model_status</code>
Histogram	Distribusi nilai dalam bucket-bucket yang sudah ditentukan	Distribusi waktu masak (0-5 mnt, 5-10 mnt, 10-15 mnt, 15-20 mnt)	<code>request_duration_seconds_bucket</code>
Summary	Percentile pre-computed di sisi client	Persentase waktu masak (P50, P95, P99)	<code>request_duration_quantiles</code>

Histogram vs Summary — kapan pakai mana?

	Histogram	Summary
Bucket/Quantile	Dihitung server-side oleh Prometheus	Dihitung client-side di aplikasi
Agregasi multi-instance	Bisa — gunakan <code>histogram_quantile()</code> di PromQL	Tidak bisa
Akurasi	Tergantung definisi bucket	Lebih akurat tapi tidak fleksibel
Rekomendasi	Gunakan hampir selalu	Hanya jika perlu akurasi tinggi, single instance

5. Implementasi — `metrics.py`

Semua metric didefinisikan terpusat di satu file. Ini adalah prinsip *Single Source of Truth* — satu tempat untuk semua definisi instrument.

```
# app/metrics.py
from prometheus_client import Counter, Histogram, Gauge

# Counter: hanya increment, tidak pernah turun
PREDICTIONS_TOTAL = Counter(
    'predictions_total',
    'Total request prediksi yang diproses',
    ['model_name', 'status'] # label = dimensi filter
)

# Histogram: distribusi latency – bucket disesuaikan workload aktual
PREDICTION_LATENCY = Histogram(
    'prediction_latency_seconds',
    'Waktu komputasi ML dalam detik',
    ['model_name'],
    buckets=[0.1, 0.5, 1.0, 2.0, 3.0, 5.0, 7.5, 10.0, 30.0]
    # PENTING: bucket harus cover worst-case workload aktual
)

# Gauge: status boolean – 1 = loaded, 0 = unloaded
MODEL_LOADED_STATUS = Gauge(
    'model_status',
    'Status model ML di memori',
```

```

        ['model_name']
    )
# Histogram untuk HTTP layer (dipakai middleware)
HTTP_REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'Durasi HTTP request dari masuk hingga keluar',
    ['method', 'path', 'status_code']
)

```

Cara Pakai di Kode Bisnis

```

# Cara 1: Manual (Opsi A)
start = time.perf_counter()
result = run_model(data)
PREDICTION_LATENCY.labels(model_name='v1').observe(time.perf_counter() - start)
# MASALAH: jika run_model() raise Exception, baris observe() tidak pernah dieksekusi
# Cara 2: Context Manager (Opsi B – DIREKOMENDASIKAN)
with PREDICTION_LATENCY.labels(model_name='v1').time():
    result = run_model(data)
# KEUNGGULAN: observe() dipanggil otomatis saat keluar from block,
# bahkan jika terjadi Exception – durasi hingga titik error tetap tercatat

```

6. Integrasi dengan RequestIDMiddleware (W15D02)

Middleware dari W15D02 diperluas untuk sekaligus melakukan instrumentasi HTTP. Satu middleware, dua fungsi: tracing (request_id) dan metrics (http duration).

```

# app/middleware.py
class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        # PENTING: skip /metrics agar scrape Prometheus tidak mencatat dirinya sendiri
        if request.url.path == '/metrics':
            return await call_next(request)

        request_id = str(uuid.uuid4())
        token = request_id_context_var.set(request_id) # dari W15D02
        start_time = time.perf_counter() # perf_counter: hardware clock, nanosecond precision

        try:
            response = await call_next(request)
            process_time = time.perf_counter() - start_time
            HTTP_REQUEST_DURATION.labels(
                method=request.method,
                path=request.url.path,
                status_code=response.status_code
            ).observe(process_time)
            response.headers['X-Request-ID'] = request_id
            return response
        except Exception as e:
            process_time = time.perf_counter() - start_time
            HTTP_REQUEST_DURATION.labels(
                method=request.method,
                path=request.url.path,
                status_code=500
            ).observe(process_time) # tetap catat meski error
            raise e
        finally:
            request_id_context_var.reset(token) # WAJIB di finally – dari W15D02

```

7. Membaca Format Output /metrics

Prometheus menggunakan format teks plain. Penting untuk bisa membaca ini karena ini adalah sumber kebenaran pertama saat debugging metric.

```
# HELP predictions_total Total request prediksi yang diproses
# TYPE predictions_total counter
predictions_total{model_name='v1',status='completed'} 1523.0
predictions_total{model_name='v1',status='error'}      12.0
# HELP prediction_latency_seconds ...
# TYPE prediction_latency_seconds histogram
prediction_latency_seconds_bucket{le='1.0',model_name='v1'} 234.0 # < 1 detik
prediction_latency_seconds_bucket{le='5.0',model_name='v1'} 891.0 # < 5 detik
prediction_latency_seconds_bucket{le='+Inf',model_name='v1'} 1523.0 # semua
prediction_latency_seconds_sum{model_name='v1'} 18432.1 # total waktu
prediction_latency_seconds_count{model_name='v1'} 1523.0 # total count
# rata-rata = _sum / _count = 18432.1 / 1523 = ~12.1 detik per prediksi
```

le = less than or equal. Bucket `le='5.0'` menampung semua request yang selesai dalam ≤ 5.0 detik. Request 5.009 detik TIDAK masuk ke bucket `le='5.0'` — dia melampaui batas, dan jatuh ke **+Inf**. Inilah mengapa bucket harus dirancang lebih besar dari ekspektasi worst-case.

8. Pitfalls yang Wajib Dihindari

Pitfall	Masalah	Fix
Bucket terlalu sempit	Semua data masuk +Inf, histogram tidak berguna	Rancang bucket berdasarkan ekspektasi aktual workload, bukan berdasarkan teori
/metrics tidak diexclude dari middleware	Setiap scrape Prometheus mencatat dirinya sendiri sebagai request	histogram method: return await call_next(request)
Duplikasi Instrumentator + custom metric	Dua metric berbeda untuk hal sama — tim bingung	Pilih satu yang jadi sumber kebenaran
observe() di luar context manager	jika exception terjadi sebelum observe(), durasi tidak tercatat	Gunakan with HISTOGRAM.labels(...).time(): untuk guarantee
setup_global_logging() di module	Worker Uvicorn saling rebut jalur logging, duplikasi handler	Gunakan logger global (misal: logging.getLogger()) — eksekusi setelah worker terisi

9. Multi-Instance & Agregasi PromQL

Di production, aplikasi berjalan dalam beberapa instance parallel (3 pod di Kubernetes, misalnya). Setiap instance menyimpan counter-nya sendiri secara terpisah — mereka tidak saling tahu. Prometheus scrape ketiga instance tersebut dan menyimpannya sebagai tiga time-series berbeda dengan label **instance** yang unik.

Untuk melihat total keseluruhan, gunakan fungsi agregasi PromQL:

```
# Total prediksi dari SEMUA instance, per model dan status
sum(predictions_total) by (model_name, status)

# Rata-rata latency p95 dari histogram (cross-instance)
histogram_quantile(0.95, sum(rate(prediction_latency_seconds_bucket[5m])) by (le, model_name))

# Ini TIDAK bisa dilakukan dengan Summary – Summary tidak bisa di-aggregate cross-instance
# Itulah mengapa Histogram lebih direkomendasikan di lingkungan distributed
```

10. Knowledge Check — Jawaban Sesi Ini

Pertanyaan	Jawaban
------------	---------

Kenapa le='5.0' bernilai 0 padahal durasi 5.009 detik?	0. Karena it strictly less than or equal. 5.009 > 5.0 ditolak bucket tersebut dan jatuh ke +Inf. Ini buktinya
Keunggulan context manager .time() vs manual time().	Exception safety. Jika run_model() raise error, .time() tetap memanggil observe() otomatis dengan context manager.
Kenapa setup_global_logging() harus di lifespan?	Jawaban: (1) worker Uvicorn sudah terisolasi, tidak ada race condition; (2) Uvicorn sudah selesai sebelum proses lain.
Apakah Counter bisa di-aggregate antar instance?	Ya. Pakai Prometheus.